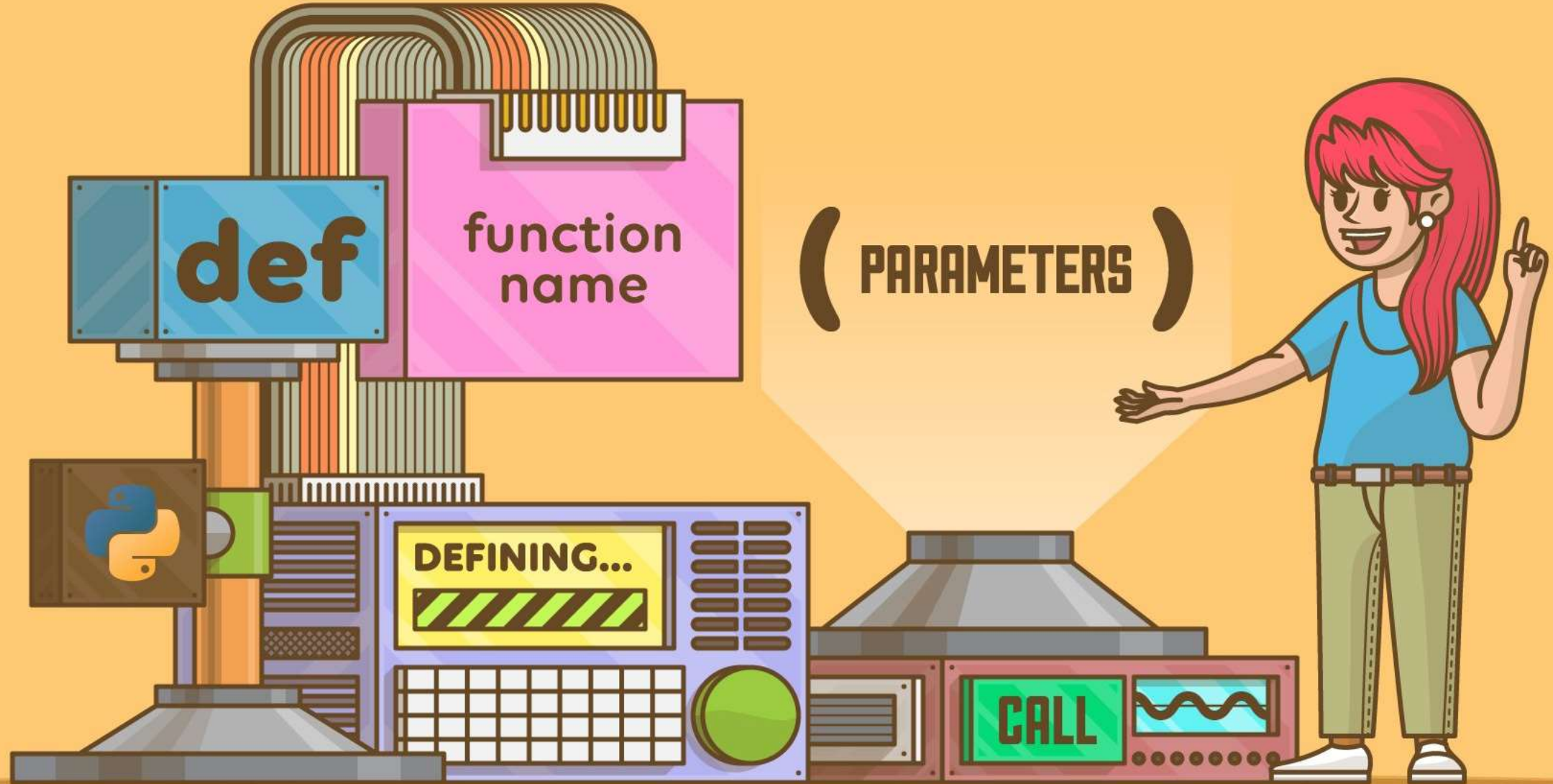


Functions in Python



➤ What is a Function?

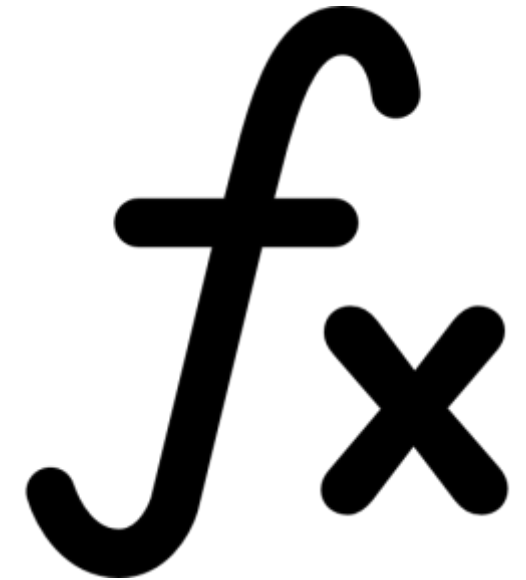
A **function** is a reusable block of code that performs a specific task.

◆ Why use functions?

- ✓ Avoid repetition
- ✓ Improve readability
- ✓ Make code modular
- ✓ Easier debugging and maintenance

□ Real-life Analogy:

Just like a washing machine washes clothes when you press a button, a function performs an action when you call it.



➤ Types of Functions

Python offers **three main types of functions**:

1 Built-in Functions ☐

These are functions **already defined in Python** — ready to use without any imports.

✦ Examples:

- **print()** – outputs text to the console
- **len()** – returns the length of a sequence
- **type()** – returns the data type of an object

```
print("Hello, World!")      # → Hello, World!
print(len("Python"))        # → 6
print(type(10))             # → <class 'int'>
```

✓ Advantages:

- Time-saving
- Optimized by Python internally
- No need to define logic



➤ Types of Functions

② User-defined Functions

Functions **created by the user** using the `def` keyword for **modularity and reusability**.

✦ Syntax:

```
def function_name(parameters):  
    # function block  
    return result
```

✦ Example:

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("Ritesh"))    # → Hello, Ritesh!
```

💡 Use Cases:

- Organizing code into reusable blocks
- Reducing repetition
- Enhancing readability





Defining a Function

```
def greet():  
    print("Hello, welcome to Python!")
```

Calling a Function

```
greet()
```

✦ Once defined, you call the function using its name followed by parentheses.

Function with Parameters

```
def greet(name):  
    print("Hello", name)  
  
greet("Ritesh")    # Function calling
```



- def is the keyword
- greet() is the function name
- Code inside the function is indented



- **Parameter** = variable in function definition
- **Argument** = actual value passed when calling



➤ Types of Functions

③ Lambda Functions ⚡

Also called **anonymous functions**, these are defined **without a name**, using the lambda keyword.

✦ Syntax:

```
lambda arguments: expression
```

✦ Example:

```
square = lambda x: x * x  
print(square(5))           # → 25
```

✦ Another example with filter():

```
nums = [1, 2, 3, 4, 5, 6]  
even_nums = list(filter(lambda x: x % 2 == 0, nums))  
print(even_nums)           # → [2, 4, 6]
```

✓ Use Cases:

- Short, throwaway functions
- Often used with map(), filter(), and reduce()



➤ Keyword and Positional Arguments

```
1  def profile(name, age):  
2      print(name, "is", age, "years old.")  
3  
4  
5  profile("Amit", 22)          # Positional  
6  profile(age=22, name="Amit") # Keyword  
7
```



Variable-Length Arguments

```
def total_marks(*marks):  
    print("Total:", sum(marks))
```

```
total_marks(85, 90, 78)
```

- ◆ *args → multiple positional args
- ◆ **kwargs → multiple keyword args



➤ Lambda (Anonymous) Functions

```
square = lambda x: x * x  
print(square(4))
```

Useful for:

- One-liners
- Sorting
- Functional programming



Nested Functions

```
def outer():  
    def inner():  
        print("Inner function")  
    inner()
```

Functions can be defined inside another function.



Scope and Lifetime of Variables

- ◆ **Local Scope** – inside a function
- ◆ **Global Scope** – outside all functions

```
x = 10  
▼ def show():  
    x = 5  
    print(x)  # Local x
```



➤ 🎯 Problem 1: Check Prime Number

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n ** 0.5)+1):  
        if n % i == 0:  
            return False  
    return True  
  
print(is_prime(29))
```



➤ 🎯 Problem 2: Fibonacci Series

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        print(a, end=" ")  
        a, b = b, a + b
```

```
fibonacci(10)
```



➤ 🎯 Problem 3: Factorial

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
print(factorial(5))
```



➤ 🎯 Problem 4: Palindrome Checker

```
def is_palindrome(s):  
    return s == s[::-1]  
  
print(is_palindrome("radar"))
```



➤ 🎯 Problem 5: Sum of Digits

```
✓ def sum_of_digits(num):  
    return sum(int(digit) for digit in str(num))  
  
print(sum_of_digits(1234))  # Output: 10
```



